

branch in the origin Git repository using the following command:

```
$ git checkout <remote-branch-name>
```

This creates a local branch with the same name as the remote branch, and which tracks the latter.

Now, here are some rules to be adhered to when working on tracking branches:

- Never work directly on any tracking branch
- Create a development branch off a tracking branch for changes that need to go into that tracking branch
- Use `$ git pull --all` in each tracking branch periodically, to keep it in sync with the *origin* repository

You can use the tracking branch as a reference to post code changes for review. For example, if the popular review-board (<https://www.reviewboard.org/>) code review tool is being used by your project team, you could use the following command:

```
$ post-review -tracking-branch=<tracking-branch-name>
```

...to post reviews for changes in that development branch.

The Git workflow

When working on a feature or a defect, it is best to adhere to the steps provided below as your development workflow. The purpose of these steps is to have a single commit addressing a single issue, which is consistent and complete in itself, while also having a clean merge history in the official repository.

At this point, we will assume that your repository reflects the latest state of the official repository. If you have a repository cloned a while ago, pull the latest changes pushed to the official repository, using the following command:

```
$ git pull --all
```

Step 1: Create a branch where you will modify code for the feature/defect assigned to you. Create the branch from the *master* branch, or from a *release* branch, to where the code must ultimately go.

Use the branch only for the specific feature or defect. Do not mix code changes for different work items in the same branch.

Step 2: Modify the code and test it.

Step 3: Commit it to the *working* branch.

Step 4: Send the code for review using the code review tool that your project uses, for example, *review-board*.

Step 5: Rework the code based on review feedback, re-test it and resend for further review. Steps 2 to 5 are repeated until review approval is obtained.

Step 6: Once review approval is received, update the *master* branch and rebase the *working* branch to get the development repository in sync with the latest code. While code changes are merged automatically most of the time, sometimes manual code merges may need to be done.

Step 7: Squash multiple commits (say, due to review feedback) into a single commit. Based on experience, I can say this is seldom done by developers. But, this is important to ensure that the Git version history looks clean and there is a single diff for a single unit of code change. This also makes it easy to cherry-pick a change into another branch.

Step 8: Send the patch to the maintainer of the official repository to incorporate your changes into the main repository.

First, ensure you are in the *master* branch. Now, you can create a branch to make the code changes. Say you are fixing *bug-ID 1234*. Then, the `<branch-name>` below can be *bug1234*.



Note: We will use the *master* branch as an example here, though it could be any other branch in the official repository where you want the code changes to go.

The few specific commands that can be used for each of the above steps are detailed. Comments are shown below in *C-style* syntax. These are for clarity only and not to be typed.

```
$ git branch <branch-name> /* Create branch */
$ git checkout <branch-name> /* Move to branch */

/* Modify code */
/* Build & test */

$ git add <files> /* Stage file for commit */
$ git commit /* Give meaningful commit log */

/* Send for review */
```

After review approval, update the *master* and rebase your branch. Rebasing is an essential step, which ensures that any changes made by other developers and pushed to the official repository (after your cloning) are visible in your branch. Any merge conflicts can be resolved by you, facilitating a clean merge by the maintainer.

```
$ git checkout master /* Move to master branch */
$ git pull --all /* Sync with latest code in origin's master branch */
$ git checkout <branch-name> /* Move back to working branch */
$ git rebase master /* Sync branch with latest code */
/* Fix merge conflicts, if any, and re-test */
```

Now, squash multiple commits into a single commit via interactive rebase.

```
$ git rebase -i HEAD-<#>
```

In the above command, `<#>` is the number of commits from HEAD to squash; for example:

```
$ git rebase -i HEAD-3
```